

Instituție de învățământ superior

Facultate / Specializare

WhereWeFishin - Fishing Experience Discovery Platform Enhanced by AI

Lucrare de licență / disertație

Absolvent: De completat
Coordonator: De completat

Mai 2026

Cuprins

1	Introducere	1
1.1	Contextul actual și motivația alegerii temei	1
1.2	Repere comparative și delimitarea problemei	2
1.3	Prezentarea generală a aplicației WhereWeFishin	2
1.4	Obiectivele lucrării și contribuțiile originale	4
1.5	Structura lucrării	4
2	Fundamente teoretice și tehnologiile utilizate	6
2.1	Arhitectura generală a sistemului	6
2.2	Modelul de date și entitățile de domeniu	7
2.3	Harta interactivă și componenta geospațială	8
2.4	Rezervări și sesiuni de pescuit	9
2.5	Autentificare, autorizare și securitate	10
2.6	Analiza imaginilor și videoclipurilor cu ajutorul inteligenței artificiale	11
2.7	Servicii auxiliare și integrarea cu sisteme externe	13
2.8	Sinteza capitolului	13
3	Analiza cerințelor și proiectarea sistemului	15
3.1	Actorii sistemului și obiectivele operaționale	15
3.2	Cerințe funcționale	16
3.3	Cerințe non-funcționale și reguli de business	17
3.4	Proiectarea de ansamblu a sistemului	19
3.5	Modelul de domeniu și relațiile dintre entități	20
3.6	Fluxuri esențiale și diagrame recomandate	21
3.7	Sinteza capitolului	22
4	Implementare și contribuții personale	23
4.1	Arhitectura implementată a soluției	23
4.2	Implementarea backend-ului	24
4.2.1	Autentificare, autorizare și controlul accesului	24
4.2.2	Locuri de pescuit, pontoane și organizarea resurselor	24
4.2.3	Workflow-ul de aplicare pentru manager	25
4.2.4	Rezervări, sesiuni de pescuit și integrarea plății	25
4.2.5	Angajați, recenzii și servicii auxiliare	26
4.3	Implementarea frontend-ului	26
4.3.1	Structura aplicației SPA și controlul accesului în interfață	26
4.3.2	Harta interactivă și explorarea locurilor de pescuit	27
4.3.3	Coșul, calendarul și integrarea Stripe în interfață	27
4.3.4	Wizard-ul de aplicare pentru manager și dashboard-ul operațional	27

4.3.5	Verificarea prin cod QR și funcțiile administrative	27
4.4	Implementarea sistemului de analiză media	28
4.4.1	Integrarea dintre backend și serviciul Python	28
4.4.2	Pipeline-ul video: detecție, tracking și post-procesare	28
4.4.3	Afișarea rezultatelor în frontend și tratarea stării asincrone	29
4.5	Sinteza capitolului	29
5	Testare și evaluare	30
5.1	Strategia de testare și infrastructura utilizată	30
5.2	Testarea unitară și validarea datelor	31
5.3	Testarea controllerelor și a comportamentului API	31
5.4	Testarea de integrare a fluxurilor critice	32
5.5	Validarea operațională a comportamentului sistemului	32
5.6	Acoperire actuală, limitări și direcții de extindere	33
5.7	Sinteza capitolului	33

Capitolul 1

Introducere

1.1 Contextul actual și motivația alegerii temei

Procesul de digitalizare a serviciilor și activităților curente a influențat în mod vizibil și domeniile recreative, în care utilizatorii așteaptă astăzi aceleași caracteristici pe care le întâlnesc în aplicațiile comerciale mature: acces rapid la informație, interfețe coerente, automatizarea fluxurilor repetitive și posibilitatea de utilizare de pe dispozitive mobile. În acest context, pescuitul recreativ reprezintă un domeniu care păstrează încă numeroase procese fragmentate. Identificarea unei locații potrivite, verificarea disponibilității, rezervarea unui ponton, confirmarea prezenței la fața locului, comunicarea cu administratorii și păstrarea unui istoric al activității sunt, de multe ori, realizate prin combinații de rețele sociale, apeluri telefonice, formulare disparate sau mesaje transmise informal.

Din punct de vedere practic, această fragmentare generează mai multe dificultăți. Utilizatorul final investește timp în căutarea manuală a informațiilor, iar datele obținute nu sunt întotdeauna actualizate. Administratorul unei locații de pescuit trebuie să gestioneze programări, personal, zone de pescuit și relația cu clienții prin instrumente separate, ceea ce crește riscul de suprapuneri, erori de organizare și comunicare întârziată. În același timp, activitatea modernă de pescuit este tot mai frecvent însoțită de conținut media, iar analiza manuală a imaginilor și înregistrărilor video nu poate oferi un răspuns scalabil atunci când se dorește extragerea automată a unor informații relevante despre capturi.

Tema aleasă este motivată tocmai de această nevoie de integrare. O aplicație software dedicată pescuitului recreativ nu mai trebuie să se limiteze la afișarea unor locații pe hartă sau la păstrarea unui jurnal personal. Într-un context actual dominat de mobilitate, servicii web și componente inteligente, este justificată dezvoltarea unei platforme capabile să reunească într-un singur sistem funcționalități de descoperire a locurilor de pescuit, rezervare, administrare operațională, verificare la fața locului și analiză automată a conținutului media. O astfel de abordare răspunde atât nevoilor utilizatorului obișnuit, cât și cerințelor de administrare ale persoanelor care gestionează activitatea din locațiile de pescuit.

Lucrarea de față pornește de la ideea că valoarea unui produs software nu este dată exclusiv de numărul funcțiilor implementate, ci mai ales de coerența cu care acestea sunt integrate într-un flux unitar. În cazul aplicației WhereWeFishin, accentul este pus pe construirea unei experiențe complete, în care harta interactivă, rezervările, administrarea pontoanelor, rolurile diferite ale utilizatorilor și analiza asistată de inteligență artificială sunt tratate ca părți ale aceluiași sistem, nu ca module izolate dezvoltate

independent.

1.2 Repere comparative și delimitarea problemei

În peisajul actual există numeroase aplicații care susțin parțial activitățile asociate pescuitului recreativ. Platforme cunoscute precum Fishbrain, FishAngler și ANGLR pun accent pe identificarea locurilor de pescuit, pe consultarea rapoartelor locale, pe jurnalizarea capturilor și pe componenta comunitară sau de urmărire a activității individuale [2, 7, 8]. O primă categorie este formată, așadar, din soluții orientate spre comunitate și jurnalizare, în care utilizatorii își înregistrează capturile, distribuie fotografiile și urmăresc observațiile altor pescari. O a doua categorie reunește soluții orientate către evidența partidelor de pescuit și păstrarea unui istoric al activității, cu accent pe date despre trasee, capturi și echipamente. Există, de asemenea, aplicații care combină elemente de cartografiere, metadate despre capturi și funcții sociale, dar care tratează mai puțin zona de administrare operațională a unui spațiu de pescuit.

Analiza comparativă la nivel funcțional arată că majoritatea aplicațiilor existente se concentrează pe experiența individuală a pescarului, pe componenta socială sau pe organizarea informațiilor despre capturi. În schimb, procese precum rezervarea controlată a unui ponton, validarea prezenței prin mecanisme digitale, separarea rolurilor dintre utilizator, angajat, manager și administrator, precum și administrarea punctuală a unei locații de pescuit sunt tratate rar într-o manieră integrată. Această observație delimitează problema abordată în prezenta lucrare: nu este urmărită doar construirea unei aplicații pentru evidența capturilor, ci proiectarea unei platforme care să acopere atât experiența utilizatorului final, cât și activitățile operaționale necesare funcționării unui ecosistem de rezervări și administrare.

Un al doilea element de diferențiere îl reprezintă integrarea analizei automate a conținutului media. Deși există aplicații care permit încărcarea de imagini sau clipuri și altele care oferă funcții de identificare asistată, puține soluții combină direct, într-un flux coerent, funcționalități de platformă web cu procesarea imaginilor și a materialelor video prin modele specializate de inteligență artificială. Din această perspectivă, problema tratată nu se reduce la crearea unei interfețe moderne, ci include și integrarea unei componente de viziune artificială într-o arhitectură software destinată utilizării curente.

Prin urmare, contribuția aplicației WhereWeFishin este poziționată la intersecția dintre platformele de servicii digitale și sistemele software asistate de inteligență artificială. Aceasta urmărește să ofere o soluție echilibrată între nevoile de acces la informație și rezervare, pe de o parte, și nevoile de analiză, verificare și administrare, pe de altă parte. Din acest motiv, studiul de față nu propune o alternativă la aplicațiile sociale existente, ci o extindere funcțională a ideii de platformă pentru pescuit prin introducerea unor mecanisme operaționale și inteligente suplimentare.

1.3 Prezentarea generală a aplicației WhereWeFishin

WhereWeFishin este o aplicație web construită în jurul ideii de platformă integrată pentru descoperirea și administrarea experiențelor de pescuit. Din punct de vedere funcțional, sistemul permite identificarea locurilor de pescuit pe o hartă interactivă, consultarea informațiilor asociate fiecărei locații, rezervarea unui ponton pentru un anumit interval

orar, vizualizarea istoricului personal și interacțiunea cu facilități dedicate analizării imaginilor și videoclipurilor încărcate de utilizator. În paralel, sistemul oferă instrumente distincte pentru personalul operațional și pentru persoanele care administrează locațiile, astfel încât fluxurile esențiale să poată fi gestionate în același mediu software.

Arhitectura aplicației este modulară și separă clar responsabilitățile principale. Interfața cu utilizatorul este realizată ca aplicație web de tip SPA bazată pe Angular, iar componenta cartografică se sprijină pe biblioteca Leaflet pentru interacțiunea cu hărți adaptate dispozitivelor mobile și elemente geospațiale [3, 10]. Componenta server este responsabilă pentru autentificare, autorizare, expunerea serviciilor REST, validarea regulilor de business și persistarea datelor și este construită ca aplicație HTTP API pe baza ASP.NET Core [4]. La nivel de stocare este utilizată o bază de date relațională gestionată printr-un strat dedicat de acces la date. Pentru funcționalitățile de analiză media este folosit un microserviciu separat, orientat către procesarea imaginilor și a materialelor video.

Din perspectiva utilizatorilor, aplicația susține mai multe roluri operaționale. Utilizatorul obișnuit poate explora locațiile disponibile, poate efectua rezervări, își poate consulta istoricul și poate încărca imagini sau videoclipuri pentru analiză. Angajatul unei locații poate valida prezența sau anumite operații curente. Managerul administrează spațiul de pescuit, pontoanele, personalul și informațiile relevante pentru activitatea curentă, iar administratorul sistemului are control extins asupra platformei, aprobă cereri și supraveghează funcționarea generală. Introducerea acestor roluri nu are doar un scop tehnic, ci reflectă o modelare mai realistă a modului în care funcționează în practică o platformă digitală care deservește atât clienți, cât și operatori.

Un flux central al aplicației este reprezentat de rezervare. Utilizatorul selectează o locație, alege un ponton și un interval de timp, iar sistemul calculează costul și generează datele necesare confirmării rezervării. În jurul acestui flux poate fi integrată și componenta de plată online, prin utilizarea infrastructurii Stripe pentru procesarea tranzacțiilor [13]. Ulterior, rezervarea poate fi asociată unui mecanism de verificare la fața locului, prin utilizarea unui cod QR și a unui token de validare. Acest tip de abordare reduce dependența de confirmări verbale sau de verificări manuale și creează premisele unui flux controlat, auditabil și repetabil.

O altă componentă importantă este cea de analiză automată a fișierelor media. Aplicația permite încărcarea de imagini și videoclipuri, iar materialele sunt trimise către o componentă software specializată în recunoașterea speciilor de pești. În implementarea actuală, această componentă valorifică modele YOLO din ecosistemul Ultralytics pentru detecție și mecanisme de tip ByteTrack pentru urmărirea obiectelor în secvențe video [14, 15]. Rezultatele procesării pot include informații sintetice despre detecțiile realizate și despre conținutul analizat. Din punct de vedere academic, această funcționalitate este relevantă deoarece pune în evidență modul în care un sistem software orientat spre utilizare practică poate integra metode de inteligență artificială fără a compromite structura generală a aplicației.

De asemenea, aplicația include o dimensiune geospațială pronunțată. Harta interactivă este folosită nu doar pentru afișarea locurilor de pescuit, ci și pentru delimitarea unor zone specifice, pentru relaționarea poziției utilizatorului cu locațiile disponibile și pentru construirea unei interfețe intuitive de explorare. Astfel, componenta de localizare devine o parte activă a logicii aplicației și nu doar un element vizual auxiliar.

1.4 Obiectivele lucrării și contribuțiile originale

Obiectivul general al lucrării este proiectarea și implementarea unei platforme software capabile să digitalizeze un set coerent de procese asociate pescuitului recreativ, prin combinarea unei interfețe web moderne, a unui backend orientat pe servicii și a unei componente de analiză asistate de inteligență artificială. În mod concret, demersul urmărește să demonstreze că o aplicație din această categorie poate depăși modelul tradițional bazat exclusiv pe prezentarea locațiilor sau pe jurnalizarea capturilor și poate oferi suport real pentru activități operaționale și de analiză.

Pentru atingerea acestui obiectiv general, sunt urmărite mai multe obiective specifice. Primul obiectiv constă în definirea unei arhitecturi software clare, în care interfața client, logica de business, persistența datelor și procesarea media să fie separate corespunzător. Al doilea obiectiv vizează modelarea unor fluxuri reale de utilizare, precum rezervarea unui ponton, verificarea unei sesiuni, administrarea resurselor și gestionarea utilizatorilor cu roluri diferite. Al treilea obiectiv constă în integrarea unei componente de analiză media bazate pe inteligență artificială într-o platformă orientată spre utilizare curentă. Nu în ultimul rând, se urmărește obținerea unei soluții extensibile, care să poată susține dezvoltări ulterioare fără schimbări majore de arhitectură.

Părțile originale evidențiate în această lucrare sunt următoarele:

1. integrarea într-o singură platformă a funcțiilor de descoperire a locurilor de pescuit, rezervare, administrare operațională și analiză media;
2. definirea unui model de roluri care separă responsabilitățile utilizatorului final, ale angajatului, ale managerului și ale administratorului;
3. utilizarea unei hărți interactive ca element central al interacțiunii, inclusiv pentru delimitarea și gestionarea zonelor relevante dintr-o locație de pescuit;
4. introducerea unui mecanism de verificare prin cod QR și token, util pentru confirmarea controlată a unor operații asociate rezervărilor;
5. conectarea aplicației web cu un microserviciu dedicat procesării imaginilor și videoclipurilor, capabil să furnizeze rezultate automate privind conținutul media încărcat;
6. organizarea soluției într-o arhitectură modulară, potrivită pentru întreținere, testare și extindere.

Valoarea acestor contribuții nu rezidă doar în existența fiecărui element în mod separat, ci mai ales în modul în care acestea sunt reunite într-o soluție unitară. O hartă interactivă, un sistem de rezervări sau un modul de analiză media pot exista individual și în alte produse software. Caracterul aplicat al proiectului este dat de integrarea lor într-un flux coerent, orientat către un domeniu concret și către scenarii de utilizare care apar în mod curent în activitatea de pescuit recreativ.

1.5 Structura lucrării

Lucrarea este organizată astfel încât să urmărească în mod progresiv trecerea de la contextul general și fundamentele teoretice la analiza, proiectarea, implementarea și

evaluarea soluției dezvoltate. După capitolul introductiv, un capitol distinct va prezenta noțiunile teoretice și tehnologiile relevante pentru tema abordată, cu accent pe concepțiile necesare înțelegerii arhitecturii aplicației, a comunicării client–server, a modelării datelor și a utilizării componentelor de inteligență artificială pentru analiza conținutului media.

În continuare, lucrarea va include un capitol dedicat analizei cerințelor și proiectării sistemului, în care vor fi descrise rolurile utilizatorilor, cazurile de utilizare principale, structura datelor și relațiile dintre componentele arhitecturale. Acest capitol va avea rolul de a justifica deciziile de proiectare și de a delimita responsabilitățile fiecărei componente implementate.

Un capitol separat va evidenția contribuțiile personale din perspectiva implementării propriu-zise. În această parte vor fi detaliate modulele realizate, fluxurile esențiale, integrarea serviciului de analiză media și soluțiile tehnice adoptate pentru problemele întâlnite pe parcursul dezvoltării. Această secțiune trebuie să reflecte în mod clar aportul propriu al autorului, atât la nivel de concepție, cât și la nivel de realizare tehnică.

Ulterior, un capitol de testare și evaluare va discuta comportamentul aplicației în scenarii relevante, funcționarea fluxurilor critice și limitele identificate. În final, lucrarea se va încheia cu un capitol de concluzii și perspective de dezvoltare, în care vor fi sintetizate rezultatele obținute și vor fi formulate direcții realiste de extindere a sistemului.

Prin această structură, lucrarea urmărește să ofere o prezentare coerentă a unui proiect software complet, fundamentat teoretic și susținut de o implementare aplicată într-un domeniu concret. Introducerea stabilește cadrul general al problemei și justifică relevanța soluției propuse, pregătind astfel analiza detaliată din capitolele următoare.

Capitolul 2

Fundamente teoretice și tehnologiile utilizate

Acest capitol are rolul de a fixa cadrul teoretic și tehnologic necesar înțelegerii aplicației WhereWeFishin. Dacă în capitolul introductiv accentul a fost pus pe motivația temei, pe relevanța actuală a domeniului și pe prezentarea generală a soluției propuse, în continuare atenția este orientată spre conceptele care susțin proiectarea și dezvoltarea efectivă a sistemului. Sunt urmărite atât noțiuni care țin de arhitectura aplicațiilor web moderne, cât și noțiuni specifice domeniului pescuitului recreativ, precum organizarea locurilor de pescuit, gestionarea rezervărilor și definirea unei sesiuni de pescuit ca unitate operațională în cadrul platformei.

Capitolul este construit astfel încât să păstreze un echilibru între explicația teoretică și raportarea la soluția dezvoltată. Scopul nu este reproducerea unor detalii de implementare la nivel de cod, ci clarificarea principiilor pe baza cărora sistemul funcționează și justificarea alegerilor tehnologice. Din această perspectivă, sunt analizate arhitectura generală a sistemului, modelul de date, componenta geospațială, fluxurile de rezervare și de validare a sesiunilor de pescuit, mecanismele de autentificare și autorizare, precum și componenta de analiză a imaginilor și videoclipurilor bazată pe inteligență artificială.

2.1 Arhitectura generală a sistemului

O aplicație software care deservește atât utilizatori finali, cât și operatori ai unei locații de pescuit are nevoie de o arhitectură clar separată pe responsabilități. În practică, un astfel de sistem trebuie să susțină simultan interacțiuni de interfață, validări de business, persistența datelor, comunicarea cu servicii externe și, în cazul de față, prelucrarea unor fișiere media. Din acest motiv, WhereWeFishin este construită ca un sistem cu mai multe componente distincte, dar integrate coerent: interfața web, componenta server, baza de date relațională și microserviciul dedicat analizei media.

La nivelul interfeței cu utilizatorul este utilizată o aplicație web de tip SPA, dezvoltată cu Angular, tehnologie potrivită pentru aplicații care necesită navigare fluidă, componente reutilizabile, formulare complexe și actualizarea dinamică a conținutului fără reîncărcarea completă a paginii [3]. În contextul proiectului analizat, această alegere este justificată de existența unor fluxuri variate, precum autentificarea, consultarea locurilor de pescuit, rezervarea pontoanelor, administrarea resurselor și încărcarea fișierelor pentru analiză. O interfață construită pe componente permite separarea clară

a responsabilităților și simplifică atât extinderea ulterioară, cât și întreținerea aplicației.

Componenta server este implementată sub forma unui API HTTP realizat cu ASP.NET Core [4]. Rolul acestei componente este de a centraliza logica de business, de a aplica regulile de validare, de a controla accesul la resurse și de a expune servicii consumabile de interfața web. Alegerea acestei platforme este justificată de suportul solid pentru dezvoltarea de servicii web, mecanismele de autentificare și autorizare, integrarea cu instrumente moderne de persistență și flexibilitatea în definirea middleware-ului și a infrastructurii necesare pentru o aplicație orientată spre servicii.

Persistența datelor este realizată într-o bază de date relațională, iar accesul la aceasta este intermediat prin Entity Framework Core, tehnologie utilizată pentru maparea dintre entitățile domeniului și tabelele bazei de date [5, 12]. Într-o aplicație precum WhereWeFishin, acest nivel este esențial deoarece stochează informațiile critice legate de utilizatori, locuri de pescuit, pontoane, sesiuni de pescuit, recenzii, repopulări cu pești și rezultate ale analizelor media. Utilizarea unui model relațional este potrivită deoarece majoritatea acestor date sunt puternic corelate și necesită integritate referențială, filtrare eficientă și interogare predictibilă.

Un element distinctiv al arhitecturii este introducerea unui microserviciu separat pentru analiza imaginilor și a videoclipurilor. Acesta este implementat în Python, utilizând Flask ca framework pentru expunerea serviciilor HTTP [9]. Separarea procesării media de API-ul principal are o justificare tehnică clară. Operațiile de inferență asupra imaginilor și videoclipurilor implică biblioteci și dependențe specializate, cerințe diferite de execuție și, uneori, acces la accelerare hardware. Menținerea lor într-un proces separat reduce cuplarea dintre componente, simplifică scalarea independentă a serviciului de analiză și diminuează riscul ca o operație costisitoare de procesare să afecteze stabilitatea generală a backend-ului responsabil de fluxurile uzuale ale aplicației.

Din punct de vedere operațional, arhitectura poate fi înțeleasă ca o succesiune de pași bine delimitați. Utilizatorul interacționează cu interfața web, aceasta trimite cereri către API-ul principal, iar API-ul aplică regulile de business și persistă datele relevante în baza de date. Atunci când este necesară analiza unui fișier media, componenta server transmite materialul către serviciul Python, primește rezultatele procesării și le stochează sub o formă utilizabilă ulterior în interfață. Această organizare corespunde principiului separării responsabilităților și contribuie la claritatea întregului sistem.

2.2 Modelul de date și entitățile de domeniu

Un sistem informatic orientat spre un domeniu concret trebuie să pornească de la o modelare corectă a entităților care definesc activitatea respectivă. În cazul unei platforme pentru pescuit recreativ, modelul de date nu descrie doar utilizatori și informații generale, ci trebuie să reflecte realități operaționale precum existența unor locuri de pescuit cu poziționare geografică precisă, a unor zone rezervabile în interiorul acestora, a unor sesiuni de pescuit programate și a unor fluxuri de validare și administrare. În acest context, proiectarea entităților are o influență directă asupra clarității aplicației și asupra posibilității de a implementa reguli de business coerente.

Entitatea centrală a domeniului este locul de pescuit. Aceasta reunește informații precum denumirea, descrierea, coordonatele geografice, imaginea asociată, tariful pe oră și legăturile cu utilizatorul care a creat locația sau cu managerul care o administrează. Prin această structură, locul de pescuit devine mai mult decât un simplu punct pe hartă.

El este un obiect de domeniu care concentrează atât date descriptive, cât și relații cu alte entități importante, precum pontoanele, recenziile, sesiunile de pescuit, angajații asociați și înregistrările privind repopularea cu pești.

În interiorul unui loc de pescuit, aplicația utilizează noțiunea de ponton. În limbajul curent al pescuitului recreativ, pontonul poate desemna o platformă sau o zonă concretă din care pescarul își desfășoară activitatea. În modelul sistemului, această entitate are și o dimensiune spațială precisă, fiind descrisă prin coordonate care delimitează zona aferentă. Prin urmare, pontonul nu este tratat doar ca un text sau o etichetă, ci ca o resursă care poate fi reprezentată pe hartă și rezervată explicit. Această alegere simplifică administrarea spațiului și permite un control mai bun al disponibilității.

Noțiunea de sesiune de pescuit ocupă un loc esențial în modelul de date. În interfața aplicației, utilizatorul percepe acest proces sub forma unei rezervări. Totuși, la nivel de model operațional, sistemul are nevoie de o entitate care să rețină utilizatorul, locul de pescuit, pontonul selectat atunci când este cazul, momentul de început, durata, prețul total și informațiile necesare validării. Din acest motiv, termenul de sesiune de pescuit este mai potrivit pentru descrierea entității interne, în timp ce termenul de rezervare descrie mai bine acțiunea inițiată de utilizator. Această distincție este importantă deoarece evită confuzia dintre gestul efectiv de a rezerva și obiectul de domeniu rezultat din acel gest.

Modelul este completat de alte entități care susțin activitatea generală a platformei. Utilizatorul este asociat cu unul sau mai multe roluri operaționale și reprezintă punctul de intrare în majoritatea fluxurilor sistemului. Recenziile oferă posibilitatea exprimării unui feedback după utilizarea unei locații. Entitățile asociate personalului permit gestionarea angajaților care lucrează într-un loc de pescuit. Repopolările sau suplimentările cu pești descriu acțiuni administrative specifice domeniului, prin care o locație își actualizează stocul biologic prin introducerea unor specii și cantități relevante. În paralel, entitățile dedicate analizelor video și analizelor imaginilor permit stocarea rezultatelor generate de componenta de inteligență artificială.

Privit în ansamblu, modelul de date răspunde unei nevoi de coerență între lumea reală și reprezentarea sa informatică. Un loc de pescuit are mai multe pontoane, un utilizator poate avea mai multe sesiuni de pescuit, iar rezultatele de analiză media trebuie să poată fi raportate la autorul încărcării și la momentul procesării. Aceste relații justifică utilizarea unei baze de date relaționale și a unui instrument ORM, deoarece ele permit exprimarea clară a dependențelor dintre obiecte și reduc riscul apariției unor inconsistențe în date. Din punct de vedere academic, această etapă de modelare este decisivă, întrucât definește vocabularul intern al sistemului și influențează toate capitolele ulterioare dedicate proiectării și implementării.

2.3 Harta interactivă și componenta geospațială

În domeniul pescuitului recreativ, poziționarea spațială nu este un element secundar, ci unul definitoriu. Utilizatorul nu caută doar un nume de locație, ci are nevoie să înțeleagă unde se află aceasta, ce alte locuri de pescuit există în apropiere, cum este organizat spațiul și ce zone sunt disponibile. Din acest motiv, componenta geospațială trebuie tratată ca o parte centrală a sistemului și nu ca o simplă completare vizuală.

Aplicația utilizează o hartă interactivă bazată pe Leaflet, bibliotecă consacrată pentru construirea de hărți web ușor de integrat și de adaptat la dispozitive mobile [10]. O astfel

de tehnologie este adecvată pentru proiect deoarece oferă suport pentru marcarea locațiilor, afișarea de ferestre informative, desenarea și stilizarea zonelor de interes, gestionarea interacțiunilor de tip click și actualizarea dinamică a elementelor de pe hartă. În plus, modul de lucru al unei biblioteci de acest tip este compatibil cu cerințele unei aplicații moderne în care datele sunt încărcate dinamic de la server.

Reprezentarea geografică pornește de la coordonate de tip latitudine și longitudine. Acestea sunt suficiente pentru afișarea locurilor de pescuit ca puncte sau zone pe hartă și oferă o bază obiectivă pentru operații precum centrare, schimbarea nivelului de zoom sau compararea pozițiilor. În cazul proiectului de față, locul de pescuit este descris prin coordonate centrale, iar pontonul poate fi reprezentat printr-o delimitare spațială mai precisă. Din punct de vedere funcțional, această alegere permite utilizatorului să înțeleagă mai ușor diferența dintre locația generală și zona exactă rezervabilă.

Componenta geospațială este utilă în cel puțin două categorii de scenarii. Prima categorie este cea de explorare, în care utilizatorul obișnuit caută un loc de pescuit potrivit, consultă poziția acestuia pe hartă și decide dacă dorește să rezerve o anumită zonă. A doua categorie este cea de administrare, în care managerul unei locații delimitează sau actualizează zona unui ponton, definește vizual resursele disponibile și controlează modul în care acestea apar în interfața publică. În ambele situații, harta devine un instrument de lucru efectiv, nu doar un element decorativ.

Un aspect important este legătura dintre reprezentarea spațială și modelul de domeniu. În WhereWeFishin, pontonul nu este tratat exclusiv ca o etichetă rezervabilă, ci și ca o zonă cu poziționare explicită. Această alegere are consecințe practice directe. Utilizatorul poate înțelege mai bine ce rezervă, iar administratorul poate controla mai precis împărțirea spațiului. Într-un domeniu în care proximitatea dintre zone și organizarea pe mal, lac sau baltă pot influența decizia utilizatorului, o astfel de reprezentare este justificată atât tehnic, cât și funcțional.

Din perspectivă teoretică, integrarea unei componente geospațiale într-o aplicație web aduce și cerințe suplimentare privind uzabilitatea. Harta trebuie să răspundă bine pe dispozitive mobile, să permită identificarea rapidă a locațiilor relevante și să nu încarce inutil interfața. De aceea, alegerea unei biblioteci ușoare și bine documentate, împreună cu structurarea clară a datelor spațiale, contribuie la obținerea unei experiențe coerente. În cazul de față, componenta geospațială susține direct scopul aplicației, deoarece ajută utilizatorul să descopere, să compare și să rezerve locuri de pescuit într-un mod intuitiv.

2.4 Rezervări și sesiuni de pescuit

Fluxul de rezervare reprezintă unul dintre procesele centrale ale aplicației, deoarece transformă informația statică despre un loc de pescuit într-o interacțiune concretă dintre utilizator și platformă. Din punct de vedere conceptual, rezervarea poate fi definită ca acțiunea prin care utilizatorul solicită utilizarea unui loc sau a unui ponton pentru un interval de timp determinat. La nivel intern, rezultatul acestei acțiuni este materializat printr-o sesiune de pescuit, entitate care reține toate datele necesare pentru gestionarea operațională a procesului.

Această distincție dintre rezervare și sesiune de pescuit este utilă atât din punct de vedere tehnic, cât și din punct de vedere al limbajului folosit în lucrare. În limbajul curent, utilizatorul afirmă că face o rezervare pentru o partidă de pescuit. În modelul aplicației,

Însă, sistemul trebuie să urmărească un obiect bine definit, care leagă utilizatorul de locul de pescuit, de un eventual ponton, de momentul de început, de durata rezervată, de cost și de starea curentă a procesului. Din acest motiv, termenul de sesiune de pescuit este mai precis atunci când este descris modelul de date sau fluxul intern al sistemului.

Pentru ca o sesiune de pescuit să poată fi creată în mod corect, sistemul trebuie să verifice mai multe condiții. Este necesar să existe o locație validă, iar în cazul în care rezervarea vizează un anumit ponton, acesta trebuie să aparțină locului de pescuit selectat. În plus, momentul de început și durata trebuie să poată fi evaluate unitar, astfel încât să fie evitate suprapunerile sau rezervările incoerente din punct de vedere temporal. Tratarea consecventă a datei și orei este importantă mai ales într-o aplicație în care utilizatorii pot interacționa de pe dispozitive diferite și din contexte diferite.

Un alt element important este calculul costului. În modelul analizat, prețul este raportat la durata utilizării și la tariful asociat locului de pescuit. Această abordare este ușor de înțeles pentru utilizator și simplifică justificarea valorii totale reținute în sesiunea de pescuit. Din punct de vedere teoretic, includerea costului calculat în entitatea finală este utilă deoarece păstrează istoricul operațional și permite o analiză ulterioară a rezervărilor fără a depinde exclusiv de valoarea curentă a tarifului din locație.

În scenariile în care aplicația folosește plată online, rezervarea poate fi completată prin integrarea unei infrastructuri externe de procesare a plăților, precum Stripe [13]. Introducerea unui astfel de serviciu are relevanță nu doar din punctul de vedere al funcționalității comerciale, ci și din perspectiva arhitecturii sistemului. Aplicația nu trebuie să implementeze de la zero logica sensibilă de prelucrare a tranzacțiilor, ci poate delega această responsabilitate către o platformă specializată, păstrând în același timp controlul asupra validărilor de business și asupra stării sesiunii de pescuit.

După confirmare, sesiunea de pescuit poate intra într-un flux de validare la fața locului. În proiectul WhereWeFishin, acest flux este susținut prin utilizarea unui cod QR și a unui token de verificare asociat sesiunii. Din punct de vedere teoretic, o astfel de soluție răspunde nevoii de trasabilitate și de reducere a erorilor umane. În locul unei verificări strict verbale sau al unei confirmări informale, platforma poate corela sesiunea existentă cu un identificator unic și cu o operație de validare controlată. Rezultatul este un proces mai ușor de auditat și mai simplu de gestionat de către angajați sau manageri.

Pe termen mai lung, formalizarea acestui flux are și o valoare analitică. O platformă care gestionează corect sesiunile de pescuit poate produce statistici privind gradul de ocupare, poate identifica perioadele aglomerate, poate calcula mai clar veniturile și poate susține relația dintre utilizatori și operatori printr-un istoric verificabil. Astfel, conceptul de sesiune de pescuit depășește rolul de simplă înregistrare și devine una dintre entitățile centrale prin care sistemul organizează activitatea curentă.

2.5 Autentificare, autorizare și securitate

Orice aplicație care gestionează conturi de utilizator, date personale, rezervări și operații administrative trebuie să trateze securitatea ca pe o cerință de bază. În cazul WhereWeFishin, această nevoie este accentuată de faptul că sistemul deservește categorii diferite de utilizatori și că unele operații, precum administrarea locurilor de pescuit, validarea sesiunilor sau accesul la date agregate, nu trebuie să fie disponibile tuturor în

mod egal.

Primul nivel al securității este reprezentat de autentificare, adică procesul prin care sistemul stabilește identitatea utilizatorului. În aplicațiile web moderne separate în frontend și backend, o soluție frecvent utilizată este reprezentată de tokenurile JWT, standardizate prin RFC 7519 [1]. Acest tip de mecanism este adecvat pentru un API consumat de o interfață SPA, deoarece permite transportul controlat al informațiilor esențiale despre utilizator și susține un model de autentificare fără stare păstrată direct pe server, potrivit pentru servicii REST.

Autentificarea singură nu este suficientă. După stabilirea identității, sistemul trebuie să decidă ce acțiuni sunt permise fiecărui utilizator. Aici intervine autorizarea, adică procesul de verificare a drepturilor asociate unui cont sau unei cereri. În cadrul unei aplicații precum WhereWeFishin, această separare este importantă deoarece utilizatorul obișnuit, angajatul, managerul și administratorul au responsabilități diferite. Un utilizator poate face rezervări și își poate consulta propriul istoric, însă nu trebuie să poată administra punctele de pescuit aparținând altor persoane. În mod similar, un manager trebuie să poată opera numai asupra resurselor asociate locațiilor pe care le administrează.

ASP.NET Core oferă suport explicit pentru mecanisme de autentificare și autorizare, inclusiv pentru limitarea accesului la anumite rute sau operații pe baza rolurilor și a regulilor definite în aplicație [4]. Din punct de vedere conceptual, această abordare este potrivită pentru sistemele multi-rol deoarece separă clar problema identității de problema permisiunilor. În practică, acest lucru conduce la o structură mai ușor de întreținut și la reducerea riscului de acces neautorizat la operații sensibile.

În plus față de controlul accesului, securitatea unei aplicații de acest tip presupune și validarea atentă a datelor primite, protejarea fluxurilor sensibile și delimitarea clară a interacțiunilor dintre componente. De exemplu, operațiile privind plățile, gestionarea sesiunilor de pescuit sau accesul la rezultatele analizelor media trebuie expuse doar în condiții bine definite. Din perspectivă teoretică, securitatea nu poate fi redusă la un singur mecanism, ci trebuie înțeleasă ca un ansamblu de decizii privind identitatea, permisiunile, trasabilitatea operațiilor și protejarea datelor importante.

Pentru o platformă cu roluri multiple, aceste mecanisme au și un rol funcțional. Ele nu doar restricționează accesul, ci fac posibilă existența unui flux coerent de lucru între actorii sistemului. Managerul poate administra locul de pescuit pe care îl coordonează, angajatul poate valida o sesiune în condiții controlate, iar administratorul poate superviza platforma fără a afecta fluxurile obișnuite ale utilizatorilor finali. Prin urmare, autentificarea și autorizarea nu reprezintă simple cerințe tehnice, ci condiții esențiale pentru funcționarea corectă a întregului ecosistem software.

2.6 Analiza imaginilor și videoclipurilor cu ajutorul inteligenței artificiale

Una dintre componentele care diferențiază aplicația WhereWeFishin de platformele centrate exclusiv pe rezervări sau pe jurnalizarea capturilor este integrarea unei componente de analiză media asistate de inteligență artificială. Din perspectivă teoretică, această zonă aparține domeniului viziunii artificiale, ramură a inteligenței artificiale care urmărește extragerea automată de informații relevante din imagini și secvențe

video. În contextul acestei lucrări, interesul principal este reprezentat de detectarea și identificarea speciilor de pești în fișiere încărcate de utilizator.

În analiza imaginilor statice, problema de bază este cea a detecției obiectelor. Un model de detecție trebuie să localizeze obiectul relevant în imagine și să îl asocieze unei clase. Familia de modele YOLO, foarte răspândită în domeniul detecției în timp real, tratează această problemă într-o manieră eficientă, fiind orientată spre viteză și aplicabilitate practică [14]. Pentru proiectul de față, utilizarea unei astfel de familii de modele este justificată de necesitatea obținerii unor rezultate utile într-un timp rezonabil și de compatibilitatea cu scenarii care implică atât imagini statice, cât și secvențe video.

La nivel conceptual, detecția peștilor într-o imagine înseamnă identificarea automată a unor regiuni de interes și atribuirea unor etichete precum specia sau tipul dominant detectat. Rezultatul nu constă doar într-o decizie binară de tipul „există” sau „nu există” pește în imagine, ci într-un set de informații mai bogat: poziția în imagine, încrederea asocierii și clasa estimată. O astfel de abordare este valoroasă în aplicațiile practice deoarece permite atât reprezentarea vizuală a rezultatului, cât și derivarea unor statistici relevante.

În cazul videoclipurilor, problema devine mai complexă. Dacă ar fi analizate independent toate cadrele, aceeași apariție ar putea fi contorizată de mai multe ori, iar rezultatul ar fi mai puțin util pentru interpretare. De aceea, în analiza secvențelor video este importantă nu doar detecția per cadru, ci și urmărirea obiectelor de-a lungul timpului. În proiectul WhereWeFishin, această idee este susținută prin utilizarea unui mecanism de tip ByteTrack, metodă dedicată urmăririi multiple a obiectelor în video [15]. Rolul acesteia este de a menține o asocierie coerentă între detecțiile succesive și de a reduce fragmentarea traseelor atunci când același exemplar apare în mai multe cadre.

Din perspectiva utilizatorului, această complexitate internă este mascată de un flux simplu: fișierul este încărcat, procesat și returnat sub forma unui rezultat interpretabil. Din perspectivă tehnică, însă, procesul presupune etape distincte. Videoclipul trebuie citit, cadrele trebuie analizate, detecțiile trebuie corelate temporal, iar rezultatele trebuie stocate într-o formă potrivită pentru afișare și interpretare. În această etapă intervin și biblioteci de procesare vizuală și multimedia, precum OpenCV pentru prelucrarea cadrelor și FFmpeg pentru recodarea și pregătirea fișierelor video într-un format mai ușor de livrat și redat [6, 11].

Rezultatul final al unei astfel de analize poate include mai multe categorii de date. Pe lângă fișierul media procesat, sistemul poate reține numărul total de detecții, specia dominantă, distribuția observațiilor pe categorii și date mai fine precum momentele în care apar obiectele detectate, cadrul video corespunzător și zona în care a fost localizat fiecare exemplar. Această bogăție de informație este importantă deoarece permite atât o prezentare intuitivă pentru utilizator, cât și o interpretare mai riguroasă din punct de vedere tehnic.

Totuși, analiza automată a imaginilor și videoclipurilor nu trebuie privită ca un mecanism infailibil. Performanța acestor sisteme depinde de calitatea imaginilor, de iluminare, de claritatea apei, de unghiurile de filmare, de gradul de suprapunere dintre obiecte și de nivelul de adaptare al modelului la speciile urmărite. Într-o lucrare academică este important ca această componentă să fie descrisă echilibrat: ea oferă valoare practică și diferențiere tehnologică, dar rezultatele sale trebuie înțelese în raport cu limitele normale ale sistemelor de viziune artificială.

2.7 Servicii auxiliare și integrarea cu sisteme externe

În afara componentelor principale, o platformă funcțională are nevoie și de servicii auxiliare care completează experiența utilizatorului și susțin operațiile curente. Aceste elemente nu reprezintă, de regulă, nucleul conceptual al aplicației, dar contribuie semnificativ la robustețea soluției și la transformarea ei dintr-un prototip într-un sistem utilizabil în condiții reale.

Un prim exemplu este reprezentat de integrarea cu un procesator de plăți. În măsura în care rezervările trebuie confirmate prin plată online, utilizarea unei platforme consacrate precum Stripe reduce complexitatea internă a aplicației și permite delegarea unor responsabilități sensibile către un serviciu specializat [13]. Din punct de vedere arhitectural, această decizie este benefică deoarece separă logica de business a platformei de detaliile critice ale procesării tranzacțiilor și oferă o bază mai sigură pentru extinderea ulterioară a fluxului de rezervare.

Un al doilea element auxiliar este sistemul de notificări prin e-mail. Confirmările trimise după realizarea unei rezervări sau mesajele legate de anumite operații importante contribuie la claritatea interacțiunii dintre utilizator și platformă. Din perspectivă teoretică, introducerea notificărilor are rolul de a consolida trasabilitatea operațiilor și de a reduce dependența de interfața activă în momentul respectiv. Utilizatorul nu trebuie să fie conectat permanent la aplicație pentru a afla rezultatul unei acțiuni importante.

Serviciile auxiliare mai au și un rol de integrare între module. De exemplu, o rezervare confirmată poate genera simultan o înregistrare internă, o notificare externă și o referință pentru validarea ulterioară prin cod QR. În mod similar, o analiză media finalizată trebuie să poată fi pusă la dispoziția interfeței într-o formă eficientă, ceea ce presupune nu doar calculul rezultatului, ci și gestionarea fișierelor rezultate și a adreselor prin care acestea sunt accesate. Astfel de detalii nu schimbă modelul de domeniu, dar influențează direct calitatea soluției finale.

Privite în ansamblu, aceste servicii auxiliare arată că dezvoltarea unei aplicații moderne nu presupune doar implementarea unor funcții principale, ci și integrarea atentă a unor componente complementare. Din punct de vedere academic, ele merită analizate deoarece evidențiază faptul că valoarea unui sistem software este dată și de modul în care acesta comunică cu infrastructura din jurul său și susține nevoile reale ale utilizatorilor.

2.8 Sinteza capitolului

Fundamentele prezentate în acest capitol conturează baza teoretică și tehnologică a aplicației WhereWeFishin. Arhitectura separată pe componente, modelul de date orientat spre entitățile reale ale domeniului, integrarea hărții interactive, formalizarea fluxurilor de rezervare și de validare a sesiunilor de pescuit, mecanismele de securitate și componenta de analiză media bazată pe inteligență artificială nu reprezintă elemente independente, ci părți ale aceluiași sistem.

Înțelegerea acestor fundamente este importantă pentru capitolele următoare, deoarece explică de ce anumite decizii de proiectare și implementare sunt justificate și cum contribuie ele la obținerea unei platforme coerente. Pe această bază poate fi analizată, în continuare, proiectarea propriu-zisă a sistemului, cu accent pe cerințele funcționale, structura fluxurilor și modul în care soluția tehnică răspunde nevoilor identificate în

domeniul pescuitului recreativ.

Capitolul 3

Analiza cerințelor și proiectarea sistemului

Acest capitol are rolul de a transforma fundamentele teoretice și tehnologice prezentate anterior într-un model coerent de cerințe și decizii de proiectare pentru aplicația WhereWeFishin. Dacă în capitolul precedent accentul a fost pus pe conceptele care susțin arhitectura generală a soluției, în continuare atenția este orientată spre modul în care aceste concepte sunt particularizate pentru domeniul pescuitului recreativ și pentru nevoile reale ale actorilor care folosesc platforma. Scopul analizei nu este descrierea exhaustivă a implementării la nivel de cod, ci justificarea structurii sistemului prin raportare la fluxurile operaționale pe care acesta trebuie să le susțină.

Din perspectivă metodologică, capitolul este construit în două direcții complementare. Într-o primă etapă sunt identificate rolurile sistemului și cerințele funcționale și non-funcționale care rezultă din activitatea acestora. Într-o a doua etapă sunt descrise principalele decizii de proiectare, relațiile dintre componentele arhitecturale și modelul de domeniu prin care aceste cerințe pot fi transpuse într-o soluție software extensibilă. Prin această abordare, analiza cerințelor nu este tratată separat de proiectare, ci ca fundament direct al alegerilor structurale adoptate în aplicație.

3.1 Actorii sistemului și obiectivele operaționale

Funcționarea platformei WhereWeFishin presupune interacțiunea dintre mai multe categorii de utilizatori, fiecare având responsabilități și drepturi diferite. Această diferențiere este importantă nu doar din perspectiva securității, ci și din perspectiva corectitudinii modelului de domeniu. O platformă care deservește simultan clienți finali, personal operațional, administratori de locații și administratori ai întregului sistem nu poate utiliza un model unic de interacțiune fără a introduce ambiguități și fără a afecta coerența fluxurilor de lucru. Din acest motiv, proiectarea sistemului pornește de la delimitarea explicită a actorilor principali.

În forma actuală a aplicației pot fi identificate patru roluri operaționale majore:

1. **utilizatorul obișnuit**, care explorează locurile de pescuit, consultă informațiile disponibile, realizează rezervări, își urmărește istoricul, transmite recenzii și poate încărca fișiere media pentru analiză;
2. **angajatul**, care operează în interiorul unuia sau mai multor locuri de pescuit și are

acces la un set restrâns de funcții asociate activității curente, precum verificarea sesiunilor și consultarea informațiilor relevante pentru spațiul în care lucrează;

3. **managerul**, care administrează un loc de pescuit, gestionează resursele asociate acestuia, precum pontoanele, angajații și informațiile descriptive, și urmărește desfășurarea rezervărilor într-un cadru controlat;
4. **administratorul sistemului**, care are o perspectivă globală asupra platformei, aprobă sau respinge cereri, supraveghează utilizatorii și păstrează controlul asupra operațiilor cu impact asupra întregului ecosistem software.

Această structură pe roluri reflectă direct cerințele domeniului. În practică, același spațiu digital trebuie să servească atât activități orientate spre client, cât și procese interne de administrare. Din punct de vedere al analizei, rezultă două consecințe importante. Prima este necesitatea unui mecanism de autentificare și autorizare capabil să separe clar permisiunile fiecărui actor. A doua este necesitatea modelării unor fluxuri operaționale distincte, dar interconectate, astfel încât acțiunile unui actor să poată deveni intrări validate pentru activitatea altui actor. De exemplu, o rezervare inițiată de utilizator trebuie să poată fi verificată ulterior de angajat, iar datele privind un loc de pescuit administrat de manager trebuie să fie vizibile în condiții controlate pentru administrator.

Din perspectiva obiectivelor operaționale, sistemul urmărește să satisfacă simultan mai multe categorii de nevoi. Pentru utilizatorul final, prioritatea este accesul rapid la informații, disponibilitate clară, rezervare controlată și interacțiune intuitivă cu harta și cu serviciile de analiză media. Pentru personalul operațional, prioritatea este existența unor instrumente simple și verificabile prin care activitatea de teren să poată fi confirmată fără ambiguități. Pentru manager, scopul principal este controlul resurselor proprii și al relației cu clienții. Pentru administrator, obiectivul este menținerea unei platforme coerente, sigure și guvernabile. Aceste scopuri justifică împărțirea aplicației în module și trasee de acces diferențiate, atât la nivel de interfață, cât și la nivel de API.

3.2 Cerințe funcționale

Analiza funcțională a sistemului poate fi organizată pe domenii de activitate, deoarece această perspectivă permite o legătură mai clară între nevoia reală, modelul de domeniu și componentele arhitecturale care trebuie să o satisfacă. În locul unei simple enumerări de ecrane sau endpoint-uri, cerințele sunt formulate aici ca responsabilități operaționale ale platformei.

O primă categorie de cerințe vizează **gestionarea identității și a accesului**. Sistemul trebuie să permită înregistrarea utilizatorilor, autentificarea lor în condiții sigure, validarea sesiunii și recuperarea controlată a accesului în cazul uitării parolei. În paralel, platforma trebuie să distingă între roluri și să restricționeze operațiile sensibile în funcție de aceste roluri. Cerința nu se rezumă la existența unui cont, ci presupune asocierea clară dintre identitate, rol și resursele asupra cărora utilizatorul poate opera.

O a doua categorie este reprezentată de **explorarea și administrarea locurilor de pescuit**. Utilizatorul trebuie să poată vizualiza pe hartă locațiile disponibile, să consulte detalii descriptive, să observe poziționarea acestora și să compare opțiuni relevante. În același timp, managerul și administratorul trebuie să poată crea, actualiza și controla informațiile asociate acestor locații. Din această cerință rezultă necesitatea

unui model geospațial suficient de clar pentru reprezentarea pozițiilor și a unor operații de administrare prin care datele afișate public să rămână corecte și actualizate.

O a treia categorie de cerințe privește **rezervările și sesiunile de pescuit**. Sistemul trebuie să permită selectarea unui interval temporal, verificarea disponibilității și crearea unei înregistrări persistente care leagă utilizatorul de locul de pescuit și, atunci când este cazul, de un ponton distinct. Tot aici intră calculul costului, asocierea unei stări explicite a rezervării și posibilitatea anulării în condiții controlate. Din punct de vedere funcțional, rezervarea nu poate fi tratată ca o simplă solicitare transmisă către server, ci ca un flux complet care include validări temporale, evitarea suprapunerilor și eventual integrarea unui mecanism de plată.

O a patra categorie este legată de **modelarea spațiului rezervabil prin pontoane și de gestionarea personalului asociat**. În cazul unui loc de pescuit popular sau extins, utilizatorul nu rezervă doar o locație generică, ci o zonă concretă. Prin urmare, aplicația trebuie să permită definirea și administrarea unor subzone rezervabile, reprezentate explicit în raport cu locația generală. În paralel, managerul trebuie să poată asocia angajați acestor locații, astfel încât o parte dintre operațiile curente să poată fi delegate fără a extinde nejustificat privilegiile la nivelul întregii platforme.

O a cincea categorie privește **fluxul de aplicare pentru rolul de manager**. Platforma trebuie să permită unui utilizator să propună spre administrare o locație, să transmită datele necesare evaluării și să intre într-un proces controlat de aprobare sau respingere. Din punct de vedere funcțional, acest workflow este important deoarece separă clar inițiativa utilizatorului de actul formal de validare administrativă. În plus, el oferă o bază pentru trasabilitate și pentru guvernanta platformei, evitând crearea necontrolată a unor noi entități de tip loc de pescuit.

O a șasea categorie reunește **funcționalitățile de conținut generat de utilizator și de administrare contextuală**. Aici intră recenziile acordate locurilor de pescuit, evidența repopulărilor cu pești, istoricul unor evenimente relevante și alte informații care completează experiența utilizatorului și oferă context operațional managerului. Aceste funcționalități nu reprezintă doar extensii auxiliare, ci contribuie la calitatea informației disponibile și la credibilitatea ecosistemului digital.

O a șaptea categorie vizează **analiza media asistată de inteligență artificială**. Sistemul trebuie să permită încărcarea de imagini și videoclipuri, transmiterea acestora către o componentă specializată de procesare și stocarea unui rezultat interpretabil pentru utilizator. Cerința este relevantă deoarece platforma nu urmărește doar digitizarea rezervărilor, ci și valorificarea conținutului media produs de utilizatori. Din punct de vedere funcțional, această zonă implică un flux asincron, validări asupra fișierelor și o modalitate de prezentare a rezultatelor care să rămână utilă, chiar dacă procesarea durează mai mult decât un apel obișnuit de tip cerere-răspuns.

Privite împreună, aceste cerințe arată că aplicația trebuie să combine funcții de tip marketplace, administrare operațională, geolocalizare și procesare inteligentă. Tocmai această combinație justifică o proiectare modulară și o separare clară a responsabilităților între componentele sistemului.

3.3 Cerințe non-funcționale și reguli de business

În afara funcționalităților vizibile pentru utilizator, aplicația trebuie să respecte un set de cerințe non-funcționale și reguli de business care garantează consistența datelor,

securitatea operațiilor și predictibilitatea comportamentului general. Aceste cerințe au un rol esențial deoarece diferențiază o demonstrație tehnică de o soluție software capabilă să susțină utilizare reală.

Prima categorie importantă este **securitatea**. Platforma trebuie să protejeze identitatea utilizatorilor, datele asociate conturilor și operațiile sensibile, precum administrarea locurilor de pescuit, verificarea rezervărilor sau aprobarea cererilor de manager. În mod concret, această cerință presupune control riguros al accesului, delimitarea responsabilităților pe roluri și protejarea fluxurilor critice prin verificări aplicate consecvent la nivel de backend. Din perspectivă de proiectare, securitatea nu este un adaos ulterior, ci o condiție transversală a întregului sistem.

O a doua categorie este **consistența temporală și integritatea rezervărilor**. Deoarece rezervările depind în mod direct de momente de început, durate și intervale care nu trebuie să se suprapună, sistemul trebuie să trateze unitar informațiile temporale. În practică, aceasta presupune normalizarea valorilor de timp, validarea faptului că o rezervare vizează un interval viitor și verificarea conflictelor dintre sesiunile existente. Tot aici intră și păstrarea unei stări explicite a sesiunii de pescuit, astfel încât diferența dintre rezervare în așteptare, rezervare confirmată și rezervare anulată să poată fi urmărită fără ambiguități.

O a treia categorie privește **fiabilitatea fluxurilor operaționale**. O rezervare validă nu trebuie compromisă de eșecul unei notificări auxiliare, iar o analiză media care întâmpină probleme trebuie să poată fi marcată explicit ca eșuată, fără a afecta stabilitatea generală a aplicației. Din acest motiv, proiectarea trebuie să prevadă mecanisme prin care procesele secundare să fie izolate de fluxurile critice, iar erorile să poată fi înregistrate și tratate ulterior. Această cerință este importantă mai ales într-un sistem care comunică cu servicii externe, precum procesatorul de plăți sau microserviciul de analiză media.

O a patra categorie este **performanța și predictibilitatea răspunsului**. Utilizatorul trebuie să poată naviga rapid între informații, să consulte disponibilitatea unei locații fără întârzieri inutile și să primească un feedback clar pentru operațiile de lungă durată. Din acest motiv, sistemul are nevoie de mecanisme de optimizare pentru datele frecvent citite și de delimitarea clară dintre operațiile care trebuie răspunse imediat și cele care pot continua asincron. Din punct de vedere al proiectării, această cerință justifică utilizarea caching-ului pentru anumite resurse și externalizarea procesării costisitoare către servicii dedicate.

O a cincea categorie este **validarea datelor de intrare și aplicarea regulilor de business**. Nu orice combinație de date primite de la client poate fi acceptată. De exemplu, o locație trebuie să existe înainte de a putea fi rezervată, un ponton trebuie să aparțină locului de pescuit selectat, un utilizator nu trebuie să poată trimite mai multe aplicații active incompatibile pentru același proces de onboarding, iar recenziile trebuie să respecte un interval valoric clar. În zona analizelor media, fișierele trebuie să respecte restricții de tip și dimensiune pentru a evita supraîncărcarea serviciului și pentru a menține un comportament controlat al sistemului.

În plus, sistemul trebuie să respecte **cerințe de auditabilitate și trasabilitate**. Procesele cu impact asupra structurii platformei, precum aprobarea unei aplicații de manager, anularea unei sesiuni sau finalizarea unei analize, trebuie să poată fi urmărite prin stări și date asociate. Această cerință este importantă atât pentru administrare, cât și pentru interpretarea corectă a fluxurilor în capitolele ulterioare dedicate implementării și testării.

Prin urmare, cerințele non-funcționale completează în mod direct analiza funcțională. Ele impun limite, reguli și așteptări de calitate fără de care funcționalitățile descrise anterior nu ar putea fi utilizate în condiții coerente și sigure.

3.4 Proiectarea de ansamblu a sistemului

Pe baza cerințelor formulate, soluția WhereWeFishin este proiectată ca un sistem modular, organizat pe componente cu responsabilități clar separate. Această alegere răspunde atât complexității funcționale a platformei, cât și nevoii de extindere ulterioară. În locul unei aplicații monolitice în care toate fluxurile ar fi amestecate în același strat, proiectarea urmărește delimitarea clară dintre interfața cu utilizatorul, logica de business, persistența datelor și procesarea media.

La nivelul interacțiunii directe cu utilizatorul este plasată aplicația frontend, responsabilă pentru redarea interfeței, navigarea între ecrane, colectarea datelor de intrare și prezentarea informațiilor furnizate de server. Din perspectiva proiectării, acest strat trebuie să rămână orientat spre experiența utilizatorului și spre compunerea fluxurilor de interacțiune, fără a duplica regulile de business care trebuie impuse centralizat. Astfel, interfața poate filtra și ghida acțiunile, însă validările esențiale și deciziile privind accesul la resurse sunt tratate în backend.

Componenta server constituie nucleul logic al sistemului. Aici sunt implementate regulile de autorizare, validările de business, controlul fluxurilor de rezervare, administrarea locurilor de pescuit, procesarea aplicațiilor de manager și coordonarea comunicării cu serviciile auxiliare. Din punct de vedere al proiectării, backend-ul joacă rolul de mediator între cerințele actorilor și starea persistentă a sistemului. El primește solicitări de la client, le interpretează în contextul regulilor de domeniu și produce răspunsuri care pot fi utilizate coerent de interfață.

Persistența datelor este tratată separat, printr-un strat dedicat accesului la bază de date și printr-un model relațional centrat pe entitățile domeniului. Această alegere este justificată de numărul mare de relații dintre obiectele sistemului și de nevoia de integritate referențială. Un loc de pescuit poate avea mai multe pontoane, recenzii, angajați și sesiuni de pescuit, iar un utilizator poate interacționa cu sistemul în mai multe moduri, în funcție de rol. Din acest motiv, proiectarea modelului de date trebuie să susțină atât operații tranzacționale, cât și interogări suficient de clare pentru administrare și raportare.

Un element important al proiectării este separarea **serviciului de analiză media** într-o componentă distinctă. Această decizie apare direct din cerințele funcționale și non-funcționale. Procesarea imaginilor și a videoclipurilor presupune dependențe, resurse și timpi de execuție diferiți față de operațiile uzuale ale aplicației. Prin urmare, este mai potrivit ca această responsabilitate să fie izolată într-un microserviciu specializat, capabil să proceseze fișierele, să returneze rezultate și să limiteze impactul operațiilor costisitoare asupra API-ului principal.

În jurul acestor componente se află și **serviciile externe auxiliare**. Procesarea plăților, trimiterea notificărilor și servirea fișierelor rezultate din analiză nu schimbă modelul principal de domeniu, dar influențează semnificativ proiectarea de ansamblu. Din acest motiv, sistemul este conceput astfel încât aceste integrări să poată fi adăugate sau înlocuite fără a afecta structura esențială a fluxurilor de business. Această flexibilitate este importantă atât din punct de vedere tehnic, cât și din punct de vedere academic,

deoarece arată că proiectarea nu este legată rigid de o singură implementare externă.

3.5 Modelul de domeniu și relațiile dintre entități

Modelul de domeniu reprezintă nucleul conceptual al aplicației, deoarece descrie obiectele principale cu care operează sistemul și relațiile dintre ele. În cazul WhereWeFishin, această modelare trebuie să fie suficient de expresivă pentru a acoperi simultan dimensiunea geospațială, dimensiunea tranzacțională a rezervărilor și dimensiunea operațională a administrării unei locații de pescuit.

Entitatea centrală este **locul de pescuit**, care reunește informații descriptive, poziționare geografică, tarif, resurse asociate și legături către actorii care îl administrează. El constituie punctul de plecare pentru majoritatea fluxurilor importante ale aplicației. În jurul său gravitează atât date destinate utilizatorului final, precum recenzii sau informațiile publice, cât și date necesare administrării, precum angajații, pontoanele și înregistrările privind repopulările cu pești.

În relație directă cu locul de pescuit se află **pontonul**, modelat ca resursă rezervabilă distinctă. Introducerea acestei entități răspunde nevoii de granularitate spațială. Utilizatorul nu rezervă întotdeauna întreaga locație, ci adesea o zonă clar delimitată din interiorul acesteia. Prin urmare, pontonul devine o extensie naturală a modelului geospațial și un element esențial pentru verificarea disponibilității și pentru reprezentarea vizuală a spațiului în interfață.

Sesiunea de pescuit este entitatea care concretizează fluxul de rezervare. Ea leagă utilizatorul de o locație, de un ponton atunci când este cazul, de un interval temporal, de un cost și de o stare operațională. Din perspectiva modelului de domeniu, această entitate are un rol mai precis decât termenul generic de rezervare, deoarece reprezintă obiectul persistent asupra căruia sistemul poate aplica validări, anulări, verificări și interpretări istorice. Introducerea explicită a unei stări, precum *Pending*, *Confirmed* sau *Cancelled*, oferă trasabilitate și susține regulile de business privind conflictul temporal și ciclul de viață al unei rezervări.

Utilizatorul apare ca entitate transversală, deoarece participă în aproape toate fluxurile semnificative ale platformei. Totuși, rolul său nu este unic, ci este particularizat prin asocierea cu responsabilități diferite. Un utilizator poate iniția rezervări, poate lăsa recenzii, poate încărca fișiere pentru analiză sau poate evolua către statutul de manager printr-un proces formalizat. Din acest motiv, modelul domeniului trebuie să trateze cu atenție raportul dintre identitatea contului și tipul operațiilor permise.

Pentru zona de administrare, modelul este completat de **aplicația de manager** și de relația **angajat–loc de pescuit**. Prima descrie workflow-ul de onboarding pentru extinderea ecosistemului cu noi locații administrate. A doua permite delegarea controlată a unor activități operaționale. Aceste două elemente sunt importante deoarece arată că platforma nu gestionează doar rezervări, ci și procese instituționale interne, specifice unui sistem digital care mediază relații între clienți, operatori și administratori.

Modelul mai include **recenziile**, **înregistrările de repopulare** și **analizele media**. Recenziile sunt importante pentru feedback și pentru încrederea utilizatorului în calitatea unei locații. Înregistrările de repopulare oferă context operațional privind managementul biologic al spațiului. Analizele media introduc o dimensiune diferită a datelor, deoarece nu descriu doar relații administrative, ci și rezultate generate automat de un serviciu inteligent. În acest caz, ciclul de viață al entității trebuie să surprindă explicit stări precum

Pending, Processing, Completed și *Failed*, pentru a reflecta corect natura asincronă a fluxului de procesare.

Privite în ansamblu, aceste entități formează un model de domeniu coerent, în care obiectele principale nu sunt izolate, ci conectate prin relații care reflectă activitatea reală a platformei. Această coerență este esențială pentru capitolul următor, deoarece implementarea componentelor aplicației derivă direct din aceste relații și din regulile pe care ele le impun.

3.6 Fluxuri esențiale și diagrame recomandate

Pentru ca analiza prezentată să fie susținută și din punct de vedere vizual, capitolul trebuie completat printr-un set restrâns de diagrame care clarifică structura sistemului și fluxurile sale critice. Rolul acestor diagrame nu este de a repeta textual conținutul secțiunilor anterioare, ci de a sintetiza relațiile și succesiunile de pași care sunt mai ușor de înțeles într-o formă schematică.

Prima diagramă recomandată este **diagrama de cazuri de utilizare**, organizată în jurul celor patru roluri principale: utilizator, angajat, manager și administrator. O astfel de reprezentare ar evidenția rapid diferența dintre funcțiile orientate spre client și funcțiile orientate spre operare și guvernare. În plus, ea ar ajuta la delimitarea corectă a responsabilităților, aspect important pentru justificarea mecanismelor de autorizare.

A doua diagramă utilă este **diagrama de arhitectură pe componente**. Aceasta trebuie să prezinte frontend-ul, API-ul principal, baza de date, serviciul de analiză media și integrările externe relevante. Din punct de vedere academic, această diagramă este importantă deoarece face legătura directă dintre cerințele analizate și alegerea unei arhitecturi separate pe responsabilități.

A treia reprezentare recomandată este **diagrama modelului de date sau a relațiilor dintre entități**. În cazul acestui proiect, o astfel de diagramă este utilă pentru clarificarea poziției centrale a entității loc de pescuit, a relației sale cu pontoanele și sesiunile de pescuit și a modului în care utilizatorii și procesele administrative se leagă de aceste structuri. O astfel de sinteză ar reduce ambiguitățile dintre termenii folosiți în text și obiectele concrete ale modelului de domeniu.

În plus, sunt recomandate două **diagrame de secvență** pentru fluxurile cu impact major asupra proiectării. Prima trebuie să surprindă fluxul rezervare–plată–confirmare–verificare, incluzând actorul uman, interfața, API-ul, persistența și eventual serviciul de plată. A doua trebuie să descrie fluxul încărcare fișier–trimitere către serviciul de analiză–procesare–stocare rezultat–afișare în interfață. Aceste două reprezentări sunt relevante deoarece fac vizibile diferențele dintre un flux tranzacțional aproape imediat și un flux asincron, bazat pe procesare costisitoare.

Prin urmare, integrarea diagramelor în acest capitol nu reprezintă un element decorativ, ci o completare metodologică a analizei. Ele pot servi drept suport pentru argumentarea deciziilor de proiectare și pot pregăti mai clar cititorul pentru capitolul dedicat implementării propriu-zise.

3.7 Sinteza capitolului

Analiza realizată în acest capitol arată că platforma WhereWeFishin trebuie înțeleasă ca un sistem multi-rol, orientat simultan spre utilizatorul final, spre administrarea operațională a locurilor de pescuit și spre integrarea unor servicii auxiliare specializate. Cerințele funcționale identificate acoperă explorarea locațiilor, rezervarea și validarea sesiunilor de pescuit, administrarea resurselor, onboarding-ul managerilor și procesarea inteligentă a conținutului media. În paralel, cerințele non-funcționale impun constrângeri de securitate, consistență, fiabilitate și auditabilitate fără de care aceste fluxuri nu ar putea funcționa coerent.

Pe baza acestor cerințe, proiectarea sistemului adoptă o structură modulară, cu componente separate pentru interfață, logică de business, persistență și analiză media. Modelul de domeniu rezultat pune în centru locul de pescuit, sesiunea de pescuit și utilizatorul, completeate de entități care susțin administrarea și interpretarea activității desfășurate în platformă. Această structură oferă cadrul necesar pentru capitolul următor, în care accentul se va muta de la analiza și justificarea soluției la prezentarea modului concret în care aceasta a fost implementată.

Capitolul 4

Implementare și contribuții personale

Acest capitol descrie modul în care soluția propusă în capitolele anterioare a fost transpusă într-un sistem software funcțional. Dacă în capitolul precedent accentul a fost pus pe cerințe, reguli de business și decizii de proiectare, în continuare interesul este orientat spre implementarea propriu-zisă a componentelor principale ale platformei WhereWeFishin. Scopul acestui capitol nu este prezentarea exhaustivă a codului sursă, ci evidențierea modulelor dezvoltate, a alegerilor tehnice adoptate și a contribuțiilor personale care au făcut posibilă integrarea coerentă a unei aplicații multi-rol, geospațiale și asistate de inteligență artificială.

Din punct de vedere structural, implementarea este prezentată pe straturi tehnice. Mai întâi este analizată arhitectura implementată și modul în care frontend-ul, backend-ul, persistența și serviciul de analiză media comunică între ele. Ulterior sunt detaliate componentele backend care concentrează regulile de business, apoi componentele frontend care construiesc experiența utilizatorului și, separat, infrastructura de procesare media bazată pe Python, YOLO și ByteTrack. În interiorul fiecărei secțiuni sunt integrate explicit acele decizii care reflectă contribuția personală a autorului, atât la nivel de proiectare tehnică, cât și la nivel de implementare efectivă.

4.1 Arhitectura implementată a soluției

Implementarea aplicației WhereWeFishin urmează o structură distribuită pe componente cu responsabilități bine delimitate. Interfața utilizatorului este construită ca aplicație web de tip SPA în Angular, backend-ul este realizat ca API HTTP în ASP.NET Core, persistența este gestionată prin Entity Framework Core și o bază de date relațională, iar funcționalitățile de analiză media sunt externalizate într-un serviciu Python separat. O astfel de separare a fost adoptată pentru a permite evoluția independentă a modulelor, pentru a reduce cuplarea dintre zonele cu cerințe tehnice diferite și pentru a păstra claritatea codului în jurul unor responsabilități bine definite.

În implementarea efectivă, frontend-ul preia interacțiunea directă cu utilizatorul: autentificarea, explorarea hărții, vizualizarea locurilor de pescuit, rezervarea, administrarea și afișarea rezultatelor analizelor media. Backend-ul are rolul de a valida cererile, de a aplica regulile de business, de a impune autorizarea și de a coordona comunicarea cu sistemele externe. În spatele acestui strat se află modelul de date și depozitele de persistență, iar pentru zona de analiză media se folosește un serviciu Python separat, capabil să gestioneze inferență și post-procesare fără a încărca inutil API-ul principal.

Contribuția personală este vizibilă aici prin alegerea unei arhitecturi care separă operațiile tranzacționale uzuale de prelucrarea media costisitoare. În locul unei abordări monolitice, a fost implementată o comunicare explicită între frontend, API și serviciul de analiză, ceea ce a simplificat atât dezvoltarea incrementală, cât și depanarea fluxurilor complexe.

4.2 Implementarea backend-ului

Backend-ul reprezintă nucleul logic al aplicației și zona în care sunt concentrate majoritatea regulilor de business. Implementarea sa este construită în jurul unor controllere specializate, a unor servicii reutilizabile și a unui strat de persistență bazat pe repository-uri. În practică, această structură a permis separarea logicii de transport HTTP de logica domeniului și de accesul la date, reducând duplicarea și simplificând testarea unor comportamente cheie.

Un rol important îl are configurația generală a aplicației, unde sunt definite autentificarea JWT, limitarea dimensiunii cererilor, compresia răspunsurilor, politicile CORS, rate limiting-ul pentru endpoint-urile sensibile și clienții HTTP necesari integrării cu serviciul de analiză media. Această zonă a implementării este relevantă deoarece arată că backend-ul nu tratează doar operații CRUD, ci gestionează și preocupări transversale precum securitatea, performanța și interoperabilitatea.

4.2.1 Autentificare, autorizare și controlul accesului

Modulul de autentificare a fost implementat pentru a răspunde cerinței de separare clară a rolurilor și de protejare a operațiilor sensibile. La nivel de API, autentificarea este bazată pe tokenuri JWT, iar la nivel de cod accesul la resurse este controlat prin claims și prin reguli de autorizare reutilizabile. Alegerea acestei soluții a permis menținerea unui model coerent pentru toate tipurile de utilizatori și a redus cuplarea dintre controllere și detaliile privind identitatea curentă.

În implementarea concretă, endpoint-urile de autentificare gestionează înregistrarea, autentificarea, verificarea tokenului și fluxurile de recuperare a accesului. Rata cererilor către aceste endpoint-uri este limitată pentru a reduce riscul de abuz, iar informațiile despre utilizator sunt extrase centralizat din claims, evitând repetarea logicii în fiecare controller. O decizie importantă a fost introducerea unor extensii de autorizare pentru resursele dependente de un loc de pescuit, astfel încât verificarea dreptului de administrare să poată fi refolosită în module diferite, precum pontoanele, angajații sau stocările privind repopularea.

Contribuția personală în această zonă se reflectă în faptul că mecanismele de autorizare nu au fost tratate ca excepții locale în fiecare flux, ci au fost centralizate în extensii și reguli clare. Această organizare a făcut posibilă extinderea aplicației cu noi module fără a duplica logica de acces.

4.2.2 Locuri de pescuit, pontoane și organizarea resurselor

Implementarea locurilor de pescuit reprezintă unul dintre modulele centrale ale backend-ului, deoarece aproape toate fluxurile importante pornesc din această entitate. Controllerul dedicat gestionează listarea, filtrarea și administrarea locațiilor, iar datele sunt

interogate diferențiat în funcție de rolul utilizatorului. O decizie relevantă a fost mutarea filtrării autorizate cât mai aproape de nivelul interogării, astfel încât datele să nu fie încărcate integral și filtrate ulterior doar la nivel de răspuns.

Pentru modelarea subzonelor rezervabile a fost implementat și modulul de pontoane. Acesta permite asocierea unor zone distincte unui loc de pescuit și păstrează o legătură directă între componenta geospațială din interfață și regulile de disponibilitate din backend. În practică, pontonul devine astfel o resursă rezervabilă explicită și nu doar o informație descriptivă. Această abordare a fost importantă pentru a susține scenarii reale în care o locație conține mai multe spații distincte cu disponibilitate independentă.

Contribuția personală este vizibilă aici prin modelarea coerentă a relației dintre entitatea principală și subzonele rezervabile, precum și prin reutilizarea aceluiași reguli de autorizare pentru toate operațiile de administrare a resurselor asociate unui loc de pescuit.

4.2.3 Workflow-ul de aplicare pentru manager

Unul dintre cele mai relevante module implementate este fluxul de aplicare pentru rolul de manager. În locul unei creări directe a unui nou loc de pescuit, aplicația introduce un proces controlat în care un utilizator transmite datele unei locații propuse, iar administratorul decide aprobarea sau respingerea cererii. Implementarea acestui workflow a necesitat definirea explicită a stărilor aplicației, păstrarea istoricului de evaluare și asocierea clară dintre decizia administrativă și crearea efectivă a resursei în sistem.

La nivel de cod, controllerul dedicat gestionează crearea cererii, actualizarea ei în condiții controlate, retransmiterea după respingere și aprobarea finală. O decizie tehnică importantă a fost limitarea tranzițiilor de stare, pentru a evita inconsistențe precum editarea unei cereri deja aprobate sau dublarea cererilor active pentru același utilizator. În plus, entitatea aplicației păstrează informații despre administratorul care a evaluat cererea, momentul evaluării și motivul respingerii, ceea ce oferă trasabilitate și face fluxul auditabil.

Această zonă reprezintă o contribuție personală puternică deoarece combină model de domeniu, reguli de business și guvernanta operațională. Implementarea nu este doar tehnică, ci reflectă modul în care platforma poate crește controlat prin validarea administrativă a noilor locații.

4.2.4 Rezervări, sesiuni de pescuit și integrarea plății

Fluxul de rezervare este unul dintre cele mai dense din punct de vedere al logicii de business. La nivel de implementare, sistemul nu creează pur și simplu o înregistrare de tip rezervare, ci validează locul de pescuit, verifică disponibilitatea pentru intervalul ales, corelează opțional un ponton, calculează costul și tratează separat integrarea plății. Această structură a fost aleasă pentru a menține controlul asupra conflictelor temporale și pentru a reduce riscul de suprapuneri sau înregistrări incoerente.

O parte importantă a implementării este legătura cu Stripe. În loc de a amesteca direct logica de plată cu persistența sesiunii finale, a fost adoptată o abordare în care se generează un *PaymentIntent* care păstrează metadate relevante despre utilizator, locație și interval. Această separare oferă o bază mai sigură pentru corelarea plății cu sesiunea de pescuit și pentru evitarea unor erori de sincronizare. În paralel, modelul

intern al sesiunii normalizează temporal datele și păstrează o stare explicită, precum *Pending*, *Confirmed* sau *Cancelled*, ceea ce face posibilă urmărirea ciclului de viață al rezervării.

Contribuția personală în acest modul este dată de integrarea coerentă a regulilor de disponibilitate, a modelului de sesiune și a unei plăți externe fără a pierde consistența internă a datelor. Fluxul rezultat este unul controlat, extensibil și potrivit pentru scenarii reale de utilizare.

4.2.5 Angajați, recenzii și servicii auxiliare

Pe lângă modulele centrale, backend-ul include și componente care completează ecosistemul operațional al aplicației. Modulul pentru angajați permite asocierea utilizatorilor la locuri de pescuit, creând astfel o delegare controlată a unor operații fără a ridica privilegiile la nivel de manager sau administrator. Sistemul de recenzii introduce feedback din partea utilizatorilor și agregă informații utile pentru descoperirea locațiilor, iar în jurul acestor fluxuri sunt integrate mecanisme auxiliare precum caching-ul răspunsurilor și trimiterea de email-uri.

O decizie importantă a fost ca serviciile auxiliare să nu blocheze fluxurile critice. De exemplu, dacă trimiterea unui email eșuează, operația principală nu este anulată. În mod similar, datele des citite, precum recenziile, pot beneficia de caching și de invalidare explicită atunci când sunt modificate. Contribuția personală în această zonă constă în transformarea unor funcții aparent secundare într-o infrastructură stabilă, care îmbunătățește experiența utilizatorului fără a compromite robustețea sistemului.

4.3 Implementarea frontend-ului

Frontend-ul WhereWeFishin a fost implementat ca o aplicație Angular modernă, bazată pe componente standalone, routing cu încărcare lazy și guard-uri funcționale. Această alegere a permis organizarea interfeței în jurul unor zone funcționale clare și a simplificat separarea fluxurilor destinate utilizatorului obișnuit, managerului, angajatului și administratorului. Din punct de vedere tehnic, interfața nu este doar o reprezentare vizuală a datelor, ci un strat care orchestrează interacțiunea utilizatorului cu harta, rezervările, plățile, verificările și analiza media.

4.3.1 Structura aplicației SPA și controlul accesului în interfață

La baza implementării frontend-ului se află structura traseelor și a guard-urilor funcționale. Fiecare zonă importantă a aplicației este încărcată lazy și este protejată în funcție de rolul utilizatorului. Această organizare a redus costul inițial de încărcare și a făcut explicită corespondența dintre fluxurile din interfață și modelul de autorizare din backend. În plus, configurarea interceptorilor HTTP a permis atașarea automată a tokenului JWT și tratarea controlată a unor erori recurente, precum expirarea autentificării sau nevoia de retry.

Contribuția personală este vizibilă aici prin modul în care logica de acces a fost distribuită între rutare, guard-uri și interceptoare, astfel încât comportamentul interfeței să rămână coerent cu regulile de securitate impuse de API.

4.3.2 Harta interactivă și explorarea locurilor de pescuit

Una dintre cele mai importante componente frontend este pagina principală de explorare, construită în jurul unei hărți interactive. Implementarea acestei zone a trebuit să combine date despre locații, geolocalizarea utilizatorului, rutarea pe hartă, informațiile despre specii și interacțiunea cu elemente de tip marker și popup. Pentru aceasta au fost dezvoltate helpers dedicați pentru marker-ele personalizate, pentru stilizarea elementelor geografice și pentru construirea ferestrelor informative într-un mod reutilizabil.

Din perspectiva experienței utilizatorului, harta funcționează ca punct de intrare în platformă. Utilizatorul poate identifica rapid locațiile disponibile, poate vedea proximitatea față de poziția curentă și poate decide ce loc de pescuit merită explorat mai departe. Contribuția personală constă în integrarea coerentă a componentei Leaflet cu datele reale ale aplicației și în transformarea hărții dintr-un simplu element vizual într-un instrument central de navigare și descoperire.

4.3.3 Coșul, calendarul și integrarea Stripe în interfață

Fluxul de rezervare are o componentă frontend complexă, deoarece utilizatorul trebuie să poată selecta intervale, să observe zilele ocupate, să combine mai multe elemente în coș și să finalizeze plata într-un mod controlat. Pentru această zonă a fost implementat un calendar capabil să evidențieze perioade rezervate, să trateze intervale parțiale și să blocheze selecțiile invalide. În paralel, integrarea Stripe din interfață încarcă dinamic elementele necesare plății și menține separată logica de confirmare a tranzacției de logica afișării.

Contribuția personală este dată de rezolvarea unor probleme practice de UX și stare: menținerea unei vizualizări clare a indisponibilităților, evitarea selecțiilor incorecte și corelarea coerentă dintre formularul de rezervare și răspunsurile backend-ului privind disponibilitatea și plata.

4.3.4 Wizard-ul de aplicare pentru manager și dashboard-ul operațional

Zona de manager include două componente cu valoare tehnică ridicată. Prima este wizard-ul de aplicare, organizat pe mai mulți pași, care combină date descriptive, alegerea locației pe hartă și o revizuire finală înainte de trimitere. A doua este dashboard-ul managerului, unde sunt reunite administrarea pontoanelor, gestionarea angajaților, actualizarea datelor locației și vizualizarea unor informații operaționale.

Implementarea dashboard-ului managerului este relevantă deoarece concentrează mai multe subfluxuri într-o interfață unitară. Aici se vede legătura directă dintre modelul de date, regulile de business și reprezentarea lor în UI. Contribuția personală constă în construirea unei interfețe capabile să deservească activități operaționale reale, nu doar scenarii demonstrative, precum și în corelarea acestora cu API-urile de autorizare și administrare existente în backend.

4.3.5 Verificarea prin cod QR și funcțiile administrative

Un flux aparte implementat în frontend este verificarea sesiunilor prin cod QR. Componenta dedicată folosește camera dispozitivului pentru citirea codului și tratează payload-ul rezervării într-o formă suficient de tolerantă pentru a gestiona variații ale

structurii datelor. Acest lucru a fost important pentru a permite un flux de verificare rapid și robust în contextul utilizării pe dispozitive mobile.

În paralel, zona administrativă oferă funcții de supraveghere asupra utilizatorilor, locațiilor și aplicațiilor de manager. Chiar dacă aceste operații sunt mai apropiate de administrarea internă decât de experiența utilizatorului final, implementarea lor în aceeași aplicație frontend a necesitat o separare clară a permisiunilor și a traseelor. Contribuția personală în această zonă este dată de integrarea uniformă a mai multor tipuri de fluxuri într-o singură aplicație SPA, fără a pierde coerența navigării și a modelului de acces.

4.4 Implementarea sistemului de analiză media

Sistemul de analiză media reprezintă componenta care diferențiază cel mai puternic proiectul WhereWeFishin de o aplicație web convențională de rezervări. Implementarea sa presupune colaborarea dintre trei straturi: interfața Angular, backend-ul .NET care orchestrează fluxul și persistă rezultatele, și serviciul Python care execută efectiv analiza. Această separare a fost necesară deoarece procesarea media implică dependențe, timpi de răspuns și constrângeri hardware diferite de restul sistemului.

4.4.1 Integrarea dintre backend și serviciul Python

La nivel de backend, componenta de analiză media este coordonată de un serviciu dedicat care trimite fișierele către aplicația Python, interpretează răspunsul și actualizează entitățile persistente asociate analizelor video și imaginilor. Controller-ele expun endpoint-uri pentru încărcare, listare, detalieri și ștergere, păstrând în același timp reguli clare de autorizare pentru accesul la rezultatele personale sau administrative.

O decizie relevantă a fost introducerea unor stări explicite pentru analiză, precum *Pending*, *Processing*, *Completed* și *Failed*. Aceste stări permit urmărirea ciclului de viață al unei procesări și oferă frontend-ului o bază clară pentru polling și afișarea rezultatelor. Contribuția personală este evidentă în această orchestrare, deoarece ea transformă un apel extern către un serviciu de viziune artificială într-un flux controlat, persistent și ușor de integrat în restul aplicației.

4.4.2 Pipeline-ul video: detecție, tracking și post-procesare

În serviciul Python, analiza video pornește de la încărcarea modelului YOLO și de la configurarea tracker-ului ByteTrack. Fiecare cadru video este parcurs pentru detectarea obiectelor relevante, iar identitatea peștilor este urmărită între cadre pentru a evita contorizarea multiplă a aceluiași exemplar. În plus, sunt păstrate traiectorii vizuale ale obiectelor detectate, iar rezultatul final este reîncodat pentru a putea fi servit eficient în interfața web.

Această parte a implementării este importantă deoarece combină considerente de performanță, consistență a rezultatelor și interpretabilitate vizuală. Contribuția personală se reflectă în configurarea pragurilor de încredere, în alegerea mecanismului de trail și în legătura dintre rezultatele brute ale modelului și forma în care ele sunt stocate și afișate ulterior în aplicație.

4.4.3 Afișarea rezultatelor în frontend și tratarea stării asincrone

În frontend, modulele pentru recunoaștere video și clasificare pe imagini trebuie să gestioneze nu doar upload-ul fișierelor, ci și o stare asincronă care evoluează în timp. Din acest motiv, a fost implementat un mecanism de polling pentru verificarea rezultatelor, iar interfața tratează separat stările de așteptare, succes și eroare. Pentru zona video au fost introduse și optimizări precum încărcarea întârziată a elementelor media și paginarea listelor de rezultate, astfel încât interfața să rămână utilizabilă chiar și în prezența unor analize numeroase.

Contribuția personală este dată aici de conectarea consecventă a stărilor persistente din backend cu mecanismele de feedback din interfață. În locul unei integrări minimale de tip upload-răspuns, a fost construit un flux complet care gestionează sănătatea serviciului, așteptarea rezultatului și prezentarea sa într-o formă utilă pentru utilizator.

4.5 Sinteza capitolului

Implementarea platformei WhereWeFishin arată modul în care o soluție analizată anterior la nivel de cerințe și proiectare poate fi transpusă într-un sistem coerent, multi-strat și orientat spre utilizare reală. Backend-ul concentrează regulile de business, controlul accesului și integrarea serviciilor externe, frontend-ul construiește experiența utilizatorului și organizația fluxurilor operaționale, iar serviciul de analiză media extinde aplicația cu o componentă de viziune artificială separată, dar integrată controlat.

Contribuția personală se reflectă în toate aceste zone: în proiectarea și implementarea workflow-ului pentru manageri, în formalizarea rezervărilor și a plăților, în construirea unei interfețe geospațiale și operaționale consistente și în integrarea unui pipeline AI capabil să proceseze atât imagini, cât și videoclipuri. Pe această bază, capitolul următor poate analiza comportamentul sistemului în scenarii concrete de testare și evaluare.

Capitolul 5

Testare și evaluare

După definirea cerințelor și prezentarea implementării, următorul pas firesc în evaluarea soluției WhereWeFishin este verificarea comportamentului acesteia prin teste și prin validări operaționale. În contextul unui sistem care combină autentificare și autorizare, rezervări, administrare multi-rol, integrare cu servicii externe și procesare media, testarea nu poate fi redusă la verificări izolate ale unor metode individuale. Este necesară o abordare care să acopere atât validarea locală a regulilor de business, cât și comportamentul compus al aplicației în scenarii apropiate de utilizarea reală.

Capitolul de față este organizat în jurul infrastructurii de testare existente în proiect și al observațiilor rezultate din rularea și analiza componentelor implementate. Accentul principal este pus pe backend, unde există cea mai consistentă suită de teste automate, dar sunt discutate și gradul redus de acoperire din frontend, validările infrastructurale asociate serviciului Python și limitele actuale ale procesului de testare. În acest fel, evaluarea nu urmărește doar să confirme funcționarea unor scenarii fericite, ci și să identifice realist nivelul de maturitate tehnică atins de soluție.

5.1 Strategia de testare și infrastructura utilizată

În forma actuală a proiectului, strategia de testare este centrată pe backend-ul .NET, unde a fost construită o infrastructură clară pentru teste unitare, teste de controller și teste de integrare. Proiectul dedicat testelor utilizează framework-ul xUnit și include dependențe pentru mocking, pentru rularea aplicației în regim de test și pentru colectarea de coverage. Din această configurație rezultă o abordare în care pot fi validate atât componente individuale, cât și fluxuri complete la nivel HTTP, fără a depinde de o instanță externă a aplicației sau de o bază de date de producție.

Un rol central în această infrastructură îl are fabrica de aplicație folosită pentru testele de integrare. Aceasta pornește API-ul într-un mediu de tip *IntegrationTesting*, injectează configurări specifice pentru conexiunea la bază de date, pentru JWT și pentru integrarea cu serviciul de recunoaștere, iar înlocuirea serviciului de email cu o implementare neutră permite verificarea fluxurilor principale fără efecte externe nedorite. Folosirea unei baze de date *InMemory* pentru scenariile de integrare oferă izolare între teste și reduce costul de execuție, făcând posibilă validarea repetată a unor fluxuri complete precum înregistrarea, autentificarea sau aprobarea aplicațiilor de manager.

Din punct de vedere al contribuției tehnice, această infrastructură este importantă deoarece permite verificarea aplicației la mai multe niveluri fără a dubla artificial codul

de test. În locul unei colecții disparate de verificări locale, proiectul beneficiază de o structură de testare coerentă, în care aceeași aplicație poate fi exercitată atât prin servicii și controllere izolate, cât și prin cereri HTTP apropiate de comportamentul real.

5.2 Testarea unitară și validarea datelor

Un prim nivel de validare este oferit de testele unitare orientate spre DTO-uri și spre reguli de validare a datelor de intrare. Aceste teste verifică faptul că obiectele folosite în autentificare, rezervări, recenzii, locuri de pescuit sau pontoane respectă constrângerile declarate prin atribute de validare. Din punct de vedere practic, această zonă este importantă deoarece multe dintre regulile de business pornesc de la date corecte și complete, iar acceptarea unor obiecte invalide ar produce erori în etapele ulterioare ale fluxurilor.

În implementarea actuală, aceste teste folosesc scenarii parametrizate pentru a acoperi multiple combinații de date valide și invalide. Sunt verificate, de exemplu, lungimea și forma credențialelor de autentificare, coerența dintre parolă și confirmarea acesteia, limitele pentru durata unei rezervări, intervalul permis pentru rating sau corectitudinea coordonatelor și a prețului unui loc de pescuit. Avantajul acestei abordări este că un număr relativ redus de metode de test poate acoperi un spectru larg de cazuri-limită, fără a fragmenta inutil suita de testare.

Pe lângă validarea DTO-urilor, testele unitare vizează și servicii cu logică de business mai densă, precum autentificarea și integrarea cu serviciul de analiză video. Aceste teste urmăresc comportamentul local al componentelor, izolat de infrastructura HTTP sau de persistență, și sunt utile pentru verificarea unor decizii precum generarea răspunsului de autentificare, tratarea erorilor sau actualizarea stărilor analizelor media. Din perspectiva evaluării, ele confirmă faptul că regulile esențiale sunt verificabile și în afara fluxurilor complete de integrare.

5.3 Testarea controllerelor și a comportamentului API

Un al doilea nivel de validare este oferit de testele dedicate controllerelor. Acestea verifică în mod direct răspunsurile HTTP, codurile de stare și reacția API-ului la cereri valide sau invalide, dar fără a porni întregul sistem în forma completă a testelor de integrare. Rolul lor este de a confirma că punctele de intrare ale backend-ului respectă contractele așteptate și că tratează corect atât situațiile reușite, cât și scenariile de eroare.

Pentru aplicația WhereWeFishin, această zonă este relevantă mai ales pentru controllerele care concentrează logică operațională, precum cele pentru autentificare, rezervări, administrarea locațiilor sau analiză video. În cazul rezervărilor, de exemplu, testarea controllerului permite verificarea modului în care sunt interpretate datele de intrare, a reacției la conflicte de disponibilitate și a integrării corecte cu componente auxiliare, precum notificările. În mod similar, controllerele de analiză video pot fi validate din perspectiva accesului autorizat, a limitelor de upload și a structurii răspunsurilor trimise către client.

Din punct de vedere metodologic, existența acestui nivel intermediar de testare este importantă deoarece reduce riscul ca problemele de contract API să fie descoperite

abia în testele de integrare sau direct în utilizarea manuală. În plus, el oferă un cadru potrivit pentru verificarea controlată a claims-urilor utilizatorului și a regulilor de acces fără a fi nevoie de parcurgerea completă a tuturor fluxurilor de autentificare.

5.4 Testarea de integrare a fluxurilor critice

Zona cu cea mai mare valoare pentru evaluarea aplicației este reprezentată de testele de integrare. Acestea folosesc infrastructura comună a fabricii de aplicație și exercită API-ul prin cereri HTTP reale, validând atât răspunsurile expuse de backend, cât și efectele produse asupra datelor persistente. Din această perspectivă, testele de integrare oferă cea mai apropiată imagine de modul în care aplicația se comportă în exploatare normală.

Un prim exemplu relevant este fluxul complet de autentificare. Testele de integrare dedicate acoperă scenariul de înregistrare a unui utilizator nou, validarea persistenței sale, autentificarea ulterioară și verificarea tokenului obținut. În plus, aceeași suită tratează cazuri negative precum duplicarea username-ului sau a adresei de email, precum și transmiterea unor date invalide. Astfel, evaluarea nu se rezumă la verificarea traseului ideal, ci confirmă și robustețea mecanismelor de protecție împotriva datelor inconsistente.

Un al doilea exemplu important este workflow-ul de aplicare pentru manager. Acesta implică utilizatori cu roluri diferite și o succesiune de stări care trebuie menținute coerent. Testele de integrare dedicate acestei zone sunt valoroase deoarece validează nu doar structura răspunsurilor API, ci și corectitudinea tranzițiilor de stare și impactul deciziilor administrative asupra datelor persistente. Din punct de vedere al capitolului de evaluare, acest flux este una dintre cele mai bune dovezi că partea de business logic implementată în backend este verificată pe scenarii relevante și nu doar pe fragmente izolate de cod.

Pe aceeași linie se află și fluxurile legate de rezervări și roluri operaționale. Chiar dacă nu toate scenariile sunt acoperite prin aceeași adâncime de integrare, prezența testelor pentru booking și pentru module precum angajații sau administrarea platformei arată că sistemul este evaluat și în situațiile în care mai multe componente trebuie să colaboreze: repository-uri, servicii, autorizare și baze de date de test. Această abordare crește încrederea în faptul că aplicația răspunde coerent atunci când regulile de business sunt exercitate cap-coadă.

5.5 Validarea operațională a comportamentului sistemului

Dincolo de testele automate, evaluarea aplicației trebuie să țină cont și de mecanismele de protecție și reziliență configurate direct în infrastructura runtime. În cazul backend-ului WhereWeFishin, aceste elemente sunt vizibile în configurația principală a aplicației și oferă o formă de validare operațională complementară testării clasice.

Un prim element important este politica strictă asociată JWT-urilor. Validarea explicită a issuer-ului, a audience-ului, a semnăturii și a duratei de viață, împreună cu o fereastră de toleranță nulă pentru expirare, contribuie la un model de securitate predictibil. În paralel, endpoint-urile sensibile de autentificare sunt protejate prin rate limiting, ceea ce

reduce riscul de abuz și arată că aplicația a fost gândită și din perspectivă operațională, nu doar funcțională.

Un al doilea element relevant este legat de manipularea cererilor voluminoase și a fluxurilor media. Limitarea dimensiunii cererilor la 150 MB, împreună cu configurarea explicită a timeout-urilor și a conexiunilor, oferă un cadru de exploatare mai stabil pentru încărcarea de videoclipuri. Tot aici pot fi incluse compresia răspunsurilor, politicile de output caching și mecanismele de retry la nivelul conexiunii cu baza de date, toate contribuind la o aplicație mai robustă în condiții reale de utilizare.

Această evaluare operațională nu înlocuiește testele automate, dar completează imaginea de ansamblu. Ea arată că soluția nu a fost construită doar pentru a trece teste locale, ci și pentru a răspunde mai bine unor cerințe reale privind securitatea, traficul și reziliența infrastructurii.

5.6 Acoperire actuală, limitări și direcții de extindere

O evaluare realistă trebuie să evidențieze nu doar punctele forte ale procesului de testare, ci și limitele actuale ale acestuia. În proiectul analizat, backend-ul beneficiază de cea mai bună acoperire, datorită combinației dintre teste unitare, teste de controller și teste de integrare. În schimb, partea de frontend este reprezentată printr-o acoperire automată redusă, centrată în principal pe un serviciu specific, iar serviciul Python pentru analiză media dispune mai ales de verificări infrastructurale și de tip smoke test, nu de o suită bogată de teste funcționale automate.

Această distribuție inegală a acoperirii este de înțeles într-un proiect în care complexitatea logicii de business este concentrată în backend, însă reprezintă totuși o limitare importantă. Lipsa unei suite consistente de teste pentru interfață înseamnă că multe dintre fluxurile frontend sunt validate în practică mai ales prin integrarea cu backend-ul și prin verificări manuale. În mod similar, partea de analiză media rămâne dependentă într-o măsură mai mare de validarea experimentală și de observarea comportamentului în execuție, decât de teste unitare fine pentru fiecare etapă din pipeline.

O altă limitare este absența, în forma actuală, a unor benchmark-uri explicite de performanță sau a unor teste end-to-end complete care să includă simultan interfața, backend-ul și serviciile externe. Din acest motiv, capitolul de față poate argumenta convingător corectitudinea funcțională și robustețea arhitecturală a unor module importante, dar nu poate pretinde o evaluare exhaustivă din perspectiva scalării sau a experienței utilizatorului în condiții de sarcină ridicată.

Ca direcții de extindere, pot fi avute în vedere dezvoltarea unei suite mai consistente de teste frontend, introducerea unor teste automatizate pentru pipeline-ul Python și generarea sistematică a unor rapoarte de coverage care să poată fi discutate și cantitativ. Chiar și în lipsa acestor extinderi, infrastructura existentă oferă o bază solidă pentru validarea logicii esențiale a platformei.

5.7 Sinteza capitolului

Testarea și evaluarea soluției WhereWeFishin arată că aplicația beneficiază de o bază solidă de validare în special la nivel de backend, unde sunt acoperite atât regulile de validare a datelor, cât și comportamentul controllerelor și fluxurile de integrare cele

mai importante. Infrastructura de test construită în jurul xUnit și a fabricii de aplicație permite verificarea unor scenarii realiste, precum autentificarea completă, workflow-ul de manager application și colaborarea dintre mai multe componente ale sistemului.

În același timp, evaluarea evidențiază limitele curente ale proiectului: acoperire automată redusă în frontend, validare limitată a serviciului Python și absența unor benchmark-uri de performanță mai ample. Aceste observații nu diminuează valoarea implementării, ci oferă o imagine realistă asupra maturității tehnice atinse și asupra pașilor care pot urma. Pe această bază, capitolul final poate sintetiza contribuțiile proiectului și direcțiile sale de dezvoltare ulterioară.

Bibliografie

- [1] Json web token (jwt). <https://datatracker.ietf.org/doc/html/rfc7519>, 2015. RFC 7519. Accesat la 09 mai 2026.
- [2] Anglr. <https://anglr.com/>, 2026. Pagină oficială a platformei. Accesat la 09 mai 2026.
- [3] Angular. <https://angular.dev/>, 2026. Documentație oficială Angular. Accesat la 09 mai 2026.
- [4] Asp.net core documentation. <https://learn.microsoft.com/en-us/aspnet/core/>, 2026. Documentație oficială Microsoft pentru aplicații și servicii web ASP.NET Core. Accesat la 09 mai 2026.
- [5] Entity framework core documentation. <https://learn.microsoft.com/en-us/ef/core/>, 2026. Documentație oficială Microsoft pentru Entity Framework Core. Accesat la 09 mai 2026.
- [6] Ffmpeg documentation. <https://ffmpeg.org/documentation.html>, 2026. Documentație oficială FFmpeg. Accesat la 09 mai 2026.
- [7] Fishangler. <https://www.fishangler.com/>, 2026. Pagină oficială a platformei. Accesat la 09 mai 2026.
- [8] Fishbrain. <https://fishbrain.com/>, 2026. Pagină oficială a platformei. Accesat la 09 mai 2026.
- [9] Flask documentation. <https://flask.palletsprojects.com/>, 2026. Documentație oficială pentru framework-ul Flask. Accesat la 09 mai 2026.
- [10] Leaflet. <https://leafletjs.com/>, 2026. Bibliotecă open-source pentru hărți interactive. Accesat la 09 mai 2026.
- [11] Opencv documentation. <https://docs.opencv.org/>, 2026. Documentație oficială pentru biblioteca OpenCV. Accesat la 09 mai 2026.
- [12] Sql server documentation. <https://learn.microsoft.com/en-us/sql/sql-server/>, 2026. Documentație oficială Microsoft pentru SQL Server. Accesat la 09 mai 2026.
- [13] Stripe documentation. <https://docs.stripe.com/>, 2026. Documentație oficială pentru integrarea plăților online. Accesat la 09 mai 2026.

- [14] Ultralytics yolo documentation. <https://docs.ultralytics.com/>, 2026. Documentație oficială pentru ecosistemul Ultralytics YOLO. Accesat la 09 mai 2026.
- [15] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.